

Tugas Mandiri 13: Laporan Praktikum Mandiri 13 Machine Learning

Maryam Hasnaa' Syamila - 0110222067

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

E-mail: hasyamil4@gmail.com

Link Notebook Colab:

 Maryam Hasnaa' Syamila_0110222067_PraktikumMandiri13.ipynb

Link GitHub Tugas:

<https://github.com/maryamhasnaasyamila/machine-learning/tree/tugas/praktikum/pratikum13/>

Link GitHub Praktikum:

<https://github.com/maryamhasnaasyamila/machine-learning/tree/praktikum/praktikum/praktikum13/>

Implementasi Artificial Neural Network (MLP) untuk Klasifikasi Citra Angka Tulisan Tangan

1. Install Library Kaggle

```
1 # install librray kaggle
2 !pip install kaggle
```

```
Requirement already satisfied: kaggle in /usr/local/lib/python3.1
Requirement already satisfied: bleach in /usr/local/lib/python3.1
Requirement already satisfied: certifi<=14.05.14 in /usr/local/li
Requirement already satisfied: charset-normalizer in /usr/local/l
Requirement already satisfied: idna in /usr/local/lib/python3.12/
Requirement already satisfied: protobuf in /usr/local/lib/python3
Requirement already satisfied: python-dateutil<=2.5.3 in /usr/loc
Requirement already satisfied: python-slugify in /usr/local/lib/p
Requirement already satisfied: requests in /usr/local/lib/python3
Requirement already satisfied: setuptools<=21.0.0 in /usr/local/l
Requirement already satisfied: six>=1.10 in /usr/local/lib/python
Requirement already satisfied: text-unidecode in /usr/local/lib/p
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/
Requirement already satisfied: urllib3<=1.15.1 in /usr/local/lib/
Requirement already satisfied: webencodings in /usr/local/lib/pyt
```

Penjelasan Kode:

- `pip install kaggle` digunakan untuk meng-install library Kaggle ke dalam environment Google Colab.
- Library ini diperlukan agar bisa menjalankan perintah `kaggle datasets download`.
- Karena Colab belum menyediakan kredensial, instalasi ini menjadi langkah awal untuk akses dataset.

Penjelasan Output:

- Tampilannya menunjukkan status `Requirement already satisfied`, artinya library Kaggle sudah ada atau berhasil terpasang di sistem.
- Tidak ada error, sehingga proses instalasi dinyatakan berhasil.

2. Load API Kaggle

```
1 # sinkronisasi dengan google drive
2 from google.colab import drive
3 drive.mount('/content/drive')
4
5 # path API json di Google Drive
6 kaggle = "/content/drive/My Drive/SEMESTER 7/Machine
  Learning/praktikum13/datasets/kaggle.json"
```

Drive already mounted at /content/drive; to attempt to forcibly r

Penjelasan Kode:

- `drive.mount('/content/drive')` digunakan untuk menghubungkan Google Drive dengan Colab.
- Variabel `kaggle` berisi path lokasi file `kaggle.json` yang disimpan di Google Drive.
- File itu adalah credential API untuk login ke Kaggle lewat script.

Penjelasan Output:

- Output menunjukkan drive telah berhasil di-mount pada path `/content/drive`.
- Tidak ada error, sehingga file `kaggle.json` sudah dapat diakses oleh Colab.

3. Konfigurasi credential Kaggle

```
1 !mkdir -p ~/.kaggle
2 !cp "{kaggle}" ~/.kaggle/
3 !chmod 600 ~/.kaggle/kaggle.json
```

Penjelasan Kode:

- `mkdir -p ~/.kaggle` membuat folder `.kaggle` sebagai lokasi default credential.
- `cp "{kaggle}" ~/.kaggle/` menyalin file `kaggle.json` ke folder tersebut.
- `chmod 600` mengubah permission agar file hanya bisa diakses oleh pemilik (syarat keamanan API Kaggle).

Penjelasan Output:

- Tidak ada pesan error atau peringatan, artinya file `kaggle.json` berhasil tersalin dan tersetting.
- Credential valid → Colab siap mengakses Kaggle API.

4. Download Dataset dari Kaggle

```
1 !kaggle datasets download -d olafkrastovski/
  handwritten-digits-0-9
```

Dataset URL: <https://www.kaggle.com/datasets/olafkrastovski/handw>
License(s): CC0-1.0
Downloading handwritten-digits-0-9.zip to /content
0% 0.00/66.7M [00:00<?, ?B/s]
100% 66.7M/66.7M [00:00<00:00, 1.41GB/s]

Penjelasan Kode:

- `kaggle datasets download -d olafkrastovski/handwritten-digits-0-9` digunakan untuk mengunduh dataset tulisan tangan angka.
- `-d` menentukan nama dataset yang akan diambil dari Kaggle.

Penjelasan Output:

- Output menampilkan link dataset, ukuran file ZIP, serta progres download.
- Angka 100% menunjukkan proses download selesai tanpa error.

5. Extract Zip Dataset

```
1 from zipfile import ZipFile
2 import os
3 file_name = 'handwritten-digits-0-9.zip'
4
5 extract_folder = "datasets"
6 os.makedirs(extract_folder, exist_ok=True)
7
8 with ZipFile(file_name, 'r') as zip:
9     zip.extractall(extract_folder)
10    print('Done')
```

Done

Penjelasan Kode:

- `ZipFile(file_name, 'r')` membuka file ZIP dalam mode read.
- `extractall(extract_folder)` mengekstrak seluruh isi file ZIP ke folder `datasets`.
- `os.makedirs(..., exist_ok=True)` memastikan folder dibuat jika belum ada.

Penjelasan Output:

- Teks `Done` menandakan proses ekstraksi selesai.
- Ini berarti dataset sudah siap digunakan untuk preprocessing.

6. Cek Isi Dataset

```
1 data_0 = os.listdir('/content/datasets/0')
2 data_1 = os.listdir('/content/datasets/1')
3 data_2 = os.listdir('/content/datasets/2')
4 data_3 = os.listdir('/content/datasets/3')
5 data_4 = os.listdir('/content/datasets/4')
6 data_5 = os.listdir('/content/datasets/5')
7 data_6 = os.listdir('/content/datasets/6')
8 data_7 = os.listdir('/content/datasets/7')
9 data_8 = os.listdir('/content/datasets/8')
10 data_9 = os.listdir('/content/datasets/9')
```

```
1 len(data_0)
```

2236

Penjelasan Kode:

- `os.listdir()` membaca isi folder dataset berdasarkan label (0-9).
- Masing-masing variabel seperti `data_0`, `data_1` merepresentasikan kumpulan file gambar di setiap kelas.

Penjelasan Output:

- Contoh output `2236` menunjukkan jumlah gambar di folder angka 0.
- Artinya dataset telah diekstrak dengan benar dan gambar dapat diakses untuk training.

7. Import Library

```
1  # Import library utama
2  import os
3  import numpy as np
4  import cv2
5  import matplotlib.pyplot as plt
6  from tensorflow.keras.models import Sequential
7  from tensorflow.keras.layers import Dense, Dropout
8  from tensorflow.keras.utils import to_categorical
9  from sklearn.model_selection import train_test_split
```

Penjelasan Kode:

- `numpy`, `os`, `cv2`, dan `matplotlib` digunakan untuk manipulasi array, file, citra, dan visualisasi grafik.
- `Sequential`, `Dense`, `Dropout`, serta `to_categorical` digunakan untuk membuat model MLP.
- `train_test_split` digunakan untuk memisahkan dataset train dan test.

Penjelasan Output:

- Tidak ada output karena cell hanya mengimpor library.
- Import berhasil karena tidak muncul error.

8. Load Dataset & Preprocessing

```
1  # Path folder dataset hasil ekstraksi
2  dataset_path = "/content/datasets"
3
4  X = []  # untuk menyimpan citra
5  y = []  # untuk menyimpan label (0-9)
6
7  # Loop folder label (0 sampai 9)
8  for label in range(10):
9      folder = os.path.join(dataset_path, str(label))
10
11     for file in os.listdir(folder):
12         img_path = os.path.join(folder, file)
13
14         # baca citra grayscale
15         img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
16
17         # ubah ke ukuran standar 28x28 seperti MNIST
18         img = cv2.resize(img, (28, 28))
19
20         # invert warna (sering diperlukan untuk dataset kaggle)
21         img = cv2.bitwise_not(img)
22
23         # threshold untuk mempertegas bentuk angka
24         _, img = cv2.threshold(img, 128, 255, cv2.THRESH_BINARY)
25
26         # normalisasi pixel 0-255 → 0-1
27         img = img / 255.0
28
29         X.append(img)
30         y.append(label)
31
32 # ubah menjadi numpy
33 X = np.array(X)
34 y = np.array(y)
35
36 print("Shape dataset:", X.shape)
37 print("Total sampel:", len(X))
```

```
Shape dataset: (21555, 28, 28)
Total sampel: 21555
```

Penjelasan Kode:

- Loop membaca semua gambar per label dari folder.
- Citra diubah ke grayscale, di-resize ke 28×28, dibalik warnanya (invert), thresholding, dan dinormalisasi.
- `X.append(img)` menyimpan data citra, `y.append(label)` menyimpan labelnya.

Penjelasan Output:

- Output `Shape dataset: (21555, 28, 28)` berarti ada 21.555 gambar berukuran 28×28.
- "Total sampel: 21555" menunjukkan dataset berhasil termuat sepenuhnya.

9. One-hot encoding label + reshape

```
1 # ubah label menjadi one-hot (multi-class 10 kelas)
2 y_encoded = to_categorical(y, num_classes=10)
3
4 # flatten gambar 28x28 menjadi 784 fitur
5 X_flat = X.reshape(len(X), 28*28)
6
7 print("X shape:", X_flat.shape)
8 print("y shape:", y_encoded.shape)
```

```
X shape: (21555, 784)
y shape: (21555, 10)
```

Penjelasan Kode:

- `to_categorical` mengubah label angka menjadi vektor one-hot untuk multi-class.
- `reshape` mengubah citra 28×28 menjadi 784 fitur agar kompatibel dengan MLP.

Penjelasan Output:

- `X shape: (21555, 784)` menunjukkan ukuran input MLP.
- `y shape: (21555, 10)` berarti terdapat 10 kelas (0-9).

10. Split Data Train & Test

```
1 X_train, X_test, y_train, y_test = train_test_split(
2     X_flat,
3     y_encoded,
4     test_size=0.2,      # 80% train, 20% test
5     random_state=42,
6     shuffle=True
7 )
8
9 print("Train:", X_train.shape, y_train.shape)
10 print("Test :", X_test.shape, y_test.shape)
```

```
Train: (17244, 784) (17244, 10)
Test : (4311, 784) (4311, 10)
```

Penjelasan Kode:

- `train_test_split` membagi dataset dengan proporsi 80% train dan 20% test.
- `shuffle=True` mengacak data agar distribusi label merata.

Penjelasan Output:

- `Train: (17244, 784)` berarti ada 17.244 data training.
- `Test: (4311, 784)` berarti ada 4.311 data testing.

11. Bangun Model MLP (ANN)

```
1 model = Sequential([
2     Dense(512, activation='relu', input_shape=(784,)),
3     Dropout(0.2),
4
5     Dense(256, activation='relu'),
6     Dropout(0.2),
7
8     Dense(128, activation='relu'),
9     Dropout(0.2),
10
11    Dense(64, activation='relu'),
12
13    Dense(10, activation='softmax') # output multi-class
14 ])
15
16 model.summary()
```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
dense_14 (Dense)	(None, 512)	401,920
dropout_6 (Dropout)	(None, 512)	0
dense_15 (Dense)	(None, 256)	131,328
dropout_7 (Dropout)	(None, 256)	0
dense_16 (Dense)	(None, 128)	32,896
dropout_8 (Dropout)	(None, 128)	0
dense_17 (Dense)	(None, 64)	8,256
dense_18 (Dense)	(None, 10)	650

Total params: 575,050 (2.19 MB)
Trainable params: 575,050 (2.19 MB)
Non-trainable params: 0 (0.00 B)

Penjelasan Kode:

- Model terdiri dari beberapa Dense layer dengan ReLU activation.
- Dropout ditambahkan untuk mengurangi overfitting.
- Output menggunakan softmax karena ini multi-class.

Penjelasan Output:

- `model.summary()` menampilkan struktur layer, output shape, dan jumlah parameter.
- Total parameter $\approx 575K$ menunjukkan model cukup kompleks untuk citra numerik.

12. Kompilasi Model

```
1 model.compile(
2     optimizer='adam',
3     loss='categorical_crossentropy',
4     metrics=['accuracy']
5 )
```

Penjelasan Kode:

- Optimizer **adam** mempercepat proses training.
- **categorical_crossentropy** digunakan untuk multi-class classification.
- Metrics **accuracy** menilai performa model.

Penjelasan Output:

- Tidak ada output karena cell hanya konfigurasi.
- Model siap untuk training.

13. Training Model

```
1 history = model.fit(  
2     X_train, y_train,  
3     epochs=30,  
4     batch_size=64,  
5     validation_split=0.2,  
6     verbose=1  
7 )
```

Epoch 1/30
216/216 ————— 7s 22ms/step - accuracy: 0.2355 - loss: 2.0934 - val_accuracy: 0.4885 - val_loss: 1.5304
Epoch 2/30
216/216 ————— 4s 18ms/step - accuracy: 0.5396 - loss: 1.3745 - val_accuracy: 0.5935 - val_loss: 1.2290
Epoch 3/30
216/216 ————— 3s 15ms/step - accuracy: 0.6891 - loss: 0.9454 - val_accuracy: 0.6454 - val_loss: 1.0785
Epoch 4/30
216/216 ————— 5s 21ms/step - accuracy: 0.7858 - loss: 0.6680 - val_accuracy: 0.6593 - val_loss: 1.0528
Epoch 5/30
216/216 ————— 4s 17ms/step - accuracy: 0.8555 - loss: 0.4624 - val_accuracy: 0.6590 - val_loss: 1.0984
Epoch 6/30
216/216 ————— 3s 13ms/step - accuracy: 0.8902 - loss: 0.3561 - val_accuracy: 0.6614 - val_loss: 1.1596
Epoch 7/30
216/216 ————— 3s 13ms/step - accuracy: 0.9130 - loss: 0.2733 - val_accuracy: 0.6677 - val_loss: 1.2472
Epoch 8/30
216/216 ————— 4s 16ms/step - accuracy: 0.9318 - loss: 0.2195 - val_accuracy: 0.6671 - val_loss: 1.2688
Epoch 9/30
216/216 ————— 4s 19ms/step - accuracy: 0.9371 - loss: 0.2010 - val_accuracy: 0.6735 - val_loss: 1.3026
Epoch 10/30
216/216 ————— 3s 13ms/step - accuracy: 0.9462 - loss: 0.1652 - val_accuracy: 0.6796 - val_loss: 1.4392
Epoch 11/30
216/216 ————— 3s 14ms/step - accuracy: 0.9547 - loss: 0.1424 - val_accuracy: 0.6732 - val_loss: 1.4751
Epoch 12/30
216/216 ————— 3s 13ms/step - accuracy: 0.9567 - loss: 0.1285 - val_accuracy: 0.6747 - val_loss: 1.4936
Epoch 13/30
216/216 ————— 5s 21ms/step - accuracy: 0.9647 - loss: 0.1197 - val_accuracy: 0.6761 - val_loss: 1.5060
Epoch 14/30
216/216 ————— 3s 14ms/step - accuracy: 0.9621 - loss: 0.1265 - val_accuracy: 0.6619 - val_loss: 1.5773
Epoch 15/30
216/216 ————— 3s 14ms/step - accuracy: 0.9706 - loss: 0.1012 - val_accuracy: 0.6669 - val_loss: 1.5399
Epoch 16/30
216/216 ————— 6s 17ms/step - accuracy: 0.9700 - loss: 0.0967 - val_accuracy: 0.6782 - val_loss: 1.5731
Epoch 17/30
216/216 ————— 4s 14ms/step - accuracy: 0.9697 - loss: 0.0960 - val_accuracy: 0.6651 - val_loss: 1.6835
Epoch 18/30
216/216 ————— 3s 13ms/step - accuracy: 0.9686 - loss: 0.0974 - val_accuracy: 0.6715 - val_loss: 1.5973
Epoch 19/30
216/216 ————— 3s 13ms/step - accuracy: 0.9696 - loss: 0.0939 - val_accuracy: 0.6706 - val_loss: 1.5720
Epoch 20/30
216/216 ————— 4s 19ms/step - accuracy: 0.9744 - loss: 0.0781 - val_accuracy: 0.6657 - val_loss: 1.6767
Epoch 21/30
216/216 ————— 4s 16ms/step - accuracy: 0.9743 - loss: 0.0827 - val_accuracy: 0.6779 - val_loss: 1.6507
Epoch 22/30
216/216 ————— 3s 13ms/step - accuracy: 0.9761 - loss: 0.0747 - val_accuracy: 0.6643 - val_loss: 1.7169
Epoch 23/30
216/216 ————— 3s 13ms/step - accuracy: 0.9745 - loss: 0.0762 - val_accuracy: 0.6622 - val_loss: 1.7459
Epoch 24/30
216/216 ————— 7s 21ms/step - accuracy: 0.9719 - loss: 0.0828 - val_accuracy: 0.6677 - val_loss: 1.7392
Epoch 25/30
216/216 ————— 3s 13ms/step - accuracy: 0.9753 - loss: 0.0743 - val_accuracy: 0.6744 - val_loss: 1.8216
Epoch 26/30
216/216 ————— 3s 13ms/step - accuracy: 0.9759 - loss: 0.0761 - val_accuracy: 0.6605 - val_loss: 1.8018
Epoch 27/30
216/216 ————— 3s 14ms/step - accuracy: 0.9786 - loss: 0.0660 - val_accuracy: 0.6611 - val_loss: 1.9274
Epoch 28/30
216/216 ————— 5s 22ms/step - accuracy: 0.9781 - loss: 0.0682 - val_accuracy: 0.6703 - val_loss: 1.7746
Epoch 29/30
216/216 ————— 3s 13ms/step - accuracy: 0.9784 - loss: 0.0655 - val_accuracy: 0.6753 - val_loss: 1.8590

Penjelasan Kode:

- **model.fit()** melatih model menggunakan data training.
- **epochs=30** menentukan jumlah iterasi.
- **batch_size=64** menentukan jumlah sampel per batch.
- **validation_split=0.2** memecah sebagian train menjadi validation.

Penjelasan Output:

- Output menampilkan progress epoch, nilai accuracy, dan loss.
- Train accuracy meningkat hingga > 90%, menandakan model belajar.
- Validation accuracy stabil di sekitar 67%, indikasi performa generalisasi.

14. Evaluasi Model

```
1 loss, accuracy = model.evaluate(X_test, y_test)
2
3 print("Test Loss:", loss)
4 print("Test Accuracy:", accuracy)
```

```
135/135 ————— 1s 4ms/step - accuracy: 0.6702 - loss: 1.9702
Test Loss: 1.8641406297683716
Test Accuracy: 0.673857569694519
```

Penjelasan Kode:

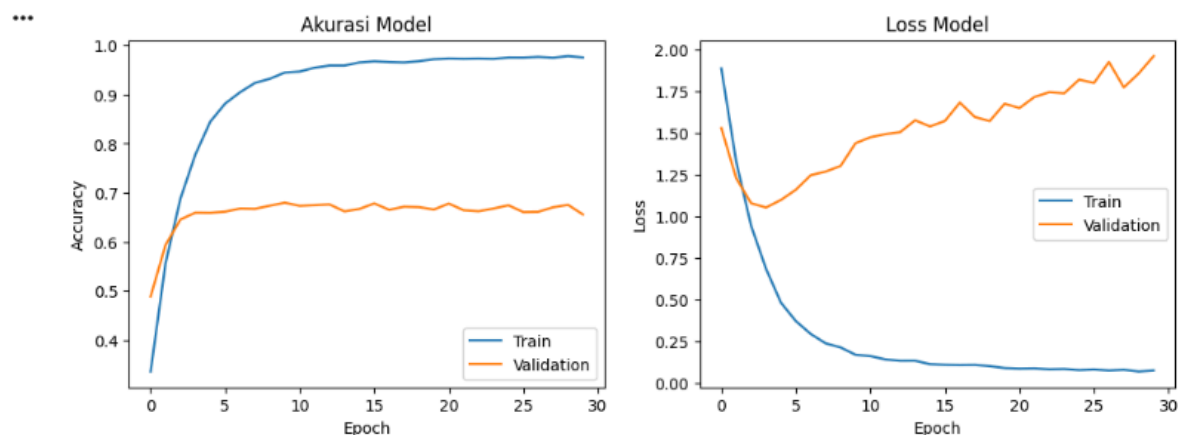
- `model.evaluate()` menguji performa model pada data test.
- Nilai loss dan accuracy dicetak ke layar.

Penjelasan Output:

- **Test Loss: 1.864...** menunjukkan error pada data test.
- **Test Accuracy: 0.6738** artinya model mampu mengenali 67% angka dengan benar.

15. Visualisasi Akurasi & Loss

```
1 plt.figure(figsize=(12,4))
2
3 # Akurasi
4 plt.subplot(1,2,1)
5 plt.plot(history.history['accuracy'], label='Train')
6 plt.plot(history.history['val_accuracy'], label='Validation')
7 plt.title("Akurasi Model")
8 plt.xlabel("Epoch")
9 plt.ylabel("Accuracy")
10 plt.legend()
11
12 # Loss
13 plt.subplot(1,2,2)
14 plt.plot(history.history['loss'], label='Train')
15 plt.plot(history.history['val_loss'], label='Validation')
16 plt.title("Loss Model")
17 plt.xlabel("Epoch")
18 plt.ylabel("Loss")
19 plt.legend()
20
21 plt.show()
```



Penjelasan Kode:

- Menggunakan matplotlib untuk memplot kurva dari history training.
- Subplot pertama menampilkan accuracy train & validation.
- Subplot kedua menampilkan loss train & validation.

Penjelasan Output:

- Kurva akurasi menunjukkan kenaikan yang tajam pada train dan stabil pada validation.
- Loss train menurun drastis namun validation cenderung meningkat, indikasi overfitting.

16. Kesimpulan Akhir

Praktikum mandiri ini berhasil mengimplementasikan Artificial Neural Network berbasis Multi-Layer Perceptron untuk melakukan klasifikasi multi-kelas pada dataset citra handwritten digits dari Kaggle. Tahapan preprocessing seperti resize, invert, thresholding, dan normalisasi terbukti membantu model dalam mengenali pola angka sehingga memperoleh akurasi pengujian sekitar 67%. Hasil evaluasi menunjukkan bahwa model mampu belajar dengan baik pada data training, meskipun masih terdapat indikasi overfitting saat diuji pada data validasi. Secara keseluruhan, praktikum ini memberikan pemahaman bahwa ANN dapat digunakan untuk pengenalan citra, namun performanya tetap terbatas dibandingkan metode yang lebih optimal untuk image seperti CNN.